Worksheet EDA1 - WS03: Structs + Ponteiros

Conteúdo: ponteiros para estruturas, operador -> e modificação indireta | Tempo previsto: 30-35 minutos | Modalidade: individual ou dupla

Objetivos de aprendizagem

- Distinguir uma variável struct de um ponteiro para struct.
- Usar corretamente os operadores . e ->.
- Rastrear alterações em campos de uma estrutura por meio de ponteiros.
- Comparar passagem de struct por valor e por ponteiro.
- Representar structs e ponteiros em diagramas de memória.

Parte A - Compreensão

Considere:

```
typedef struct
{
    int x;
    int y;
} Ponto;
Ponto a = {2, 5};
Ponto *p = &a;
```

1. Complete:
a) a é uma variável do tipo _____.
b) p é uma variável do tipo _____.
c) p armazena _____.
d) a.x acessa um campo usando o operador _____.
e) p->x acessa um campo usando o operador _____.
2. Marque V ou F e justifique as falsas:
( ) p->x e (*p).x representam o mesmo campo.
( ) a->x é a forma usual de acessar x quando a é uma variável Ponto.
( ) p.x é a forma usual de acessar x quando p é Ponto *.
( ) alterar p->y pode alterar diretamente o campo y de a.

_____

Parte B - Simulação e rastreamento

3. Considere o código abaixo. Use endereços simbólicos a@1000 e p@2000.

```
Ponto a = {2, 5};
Ponto *p = &a;
p->x = 7;
(*p).y = p->x + 1;
```

Preencha os estados:

Etapa	a.x	a.y	p aponta para
inicial			
após p->x = 7			
estado final			

4. Desenhe o estado final da memória mostrando a, p e a seta de apontamento.

_____
_____
_____

5. Agora considere:

```
Ponto a = {2, 5};
Ponto b = {1, 1};
Ponto *p = &a;
Ponto *q = &b;
q = p;
q->y = 10;
```

a) Valores finais de a: x = _____ y = _____
b) Valores finais de b: x = _____ y = _____
c) Ao final, p aponta para _____ e q aponta para _____.
d) Explique por que b.y não é alterado.

_____

Parte C - Leitura de código

6. Considere:

```
void deslocar(Ponto *p, int dx, int dy)
{
    p->x = p->x + dx;
    p->y = p->y + dy;
}
Ponto a = {3, 4};
deslocar(&a, 2, -1);
```

a) Valor final de a.x: _____
b) Valor final de a.y: _____
c) Por que a chamada usa &a?
_____

7. Compare:

```
void zerar1(Ponto p)
{
    p.x = 0;
    p.y = 0;
}
void zerar2(Ponto *p)
{
    p->x = 0;
    p->y = 0;
}
```

Para Ponto a = {4, 8};, descreva o resultado após zerar1(a) e após zerar2(&a).
_____
_____

Parte D - Construção

8. Complete a função para definir os dois campos da estrutura apontada.

```
void definir(Ponto *p, int novoX, int novoY)
{


}
```

9. Reescreva a expressão abaixo usando o operador ->.
(*p).x = (*p).y + 1;
Resposta: _____

Parte E - Problema integrador

10. Analise sem compilador:

```
typedef struct
{
    int codigo;
    int quantidade;
} Item;
void atualizar(Item *p, int valor)
{
    p->quantidade = p->quantidade + valor;
    if (p->quantidade < 0)
        p->quantidade = 0;
}
Item produto = {15, 4};
atualizar(&produto, 3);
atualizar(&produto, -10);
```

a) Valor final de produto.codigo: _____
b) Valor final de produto.quantidade: _____
c) Qual campo nunca é modificado pela função? _____
d) Represente graficamente a relação entre o parâmetro p e produto durante uma chamada.
_____
_____

Preparação para a próxima aula

Antes de sair, certifique-se de que consegue explicar:
1. quando usar . e quando usar ->;
2. por que uma função que recebe Ponto * pode modificar a estrutura do chamador;
3. como um diagrama de memória representa um ponteiro para struct.

Na próxima aula iniciaremos o estudo de organização de arquivos em C.